

Day 2 Session 2: Process-Based Parallelism

Welcome to Session 2 of Day 2! In this session, we will explore process-based parallelism in Python using the `multiprocessing` module. By the end of this session, you will know how to spawn, manage, communicate with, and synchronize processes.

1. How to Spawn a Process

To run a task in a separate process, we use the `multiprocessing.Process` class. It behaves similarly to the `threading.Thread` class but creates an entirely new operating system process.

- `Process(target=function, args=(arg1,))`: Defines the process.
- `.start()`: Spawns the process.
- `.join()`: Waits for the process to complete.

```
import multiprocessing

def worker(num):
    print(f"Worker {num} is running")

if __name__ == '__main__':
    p = multiprocessing.Process(target=worker, args=(1,))
    p.start()
    p.join()
```

2. How to Name a Process

Naming processes is useful for debugging. You can assign a name to a process during creation or access its name from within the process.

- `name` parameter in `Process`
- `process.name` attribute
- `multiprocessing.current_process().name`

3. How to Run a Process in the Background

Setting `daemon=True` marks a process as a background (daemon) process. Daemon processes are automatically terminated when the parent process exits. They are commonly used for background monitoring or heartbeats.

```
p = multiprocessing.Process(target=monitor, daemon=True)
p.start()
```

4. How to Kill a Process

Sometimes a process takes too long or encounters an error, and you need to stop it manually.

- `.terminate()`: Sends a SIGTERM signal to terminate the process cleanly.
- `.kill()`: Sends a SIGKILL signal to forcibly kill the process (Python 3.7+).
- `.is_alive()`: Returns True if the process is still running.
- `.exitcode`: Checks the exit status (0 for success).

5. How to Use a Process in a Subclass

For complex state management, you can create a custom process by subclassing `multiprocessing.Process` and overriding its `run()` method.

Now, let's see these theoretical topics in action.

- [03_process_lifecycle.py](file:///Users/vinod/Desktop/CDAC/20260927-hpc-python-parallel-programming/day2/demo/03_process_lifecycle.py)

6. Exchanging Objects Between Processes

Since processes do not share memory space, they must communicate by passing messages or objects.

Queues

A `multiprocessing.Queue` is a thread-safe and process-safe First-In-First-Out (FIFO) data structure.

```
[Producer] ----> (put) [Queue] (get) ----> [Consumer]
```

Pipes

A `multiprocessing.Pipe` provides a two-way connection between two processes. It returns two connection objects, one for each end of the pipe.

```
[Process A] <== (send/rcv) ==> [Pipe] <== (send/rcv) ==> [Process B]
```

7. Synchronizing Processes

Just like threads, processes can step on each other's toes when accessing shared resources like files or terminals. `multiprocessing` provides synchronization primitives similar to `threading`:

- **Lock**: Ensures only one process executes a critical section at a time.
- **Event**: Used for signalling between processes.
- **Semaphore**: Limits access to a resource to a fixed number of processes.
- **Barrier**: Blocks processes until a specified number of them have reached a synchronization point.

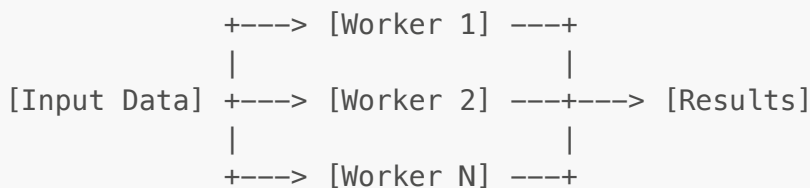
8. Managing State Between Processes

If you really need shared state, you can use:

- `multiprocessing.Value` and `multiprocessing.Array`: Shared memory primitives (requires C types).
- `multiprocessing.Manager`: Provides process-safe proxies for Python objects like dicts and lists.

9. How to Use a Process Pool

Manually spawning and joining dozens of processes can be tedious. `multiprocessing.Pool` provides a convenient way to manage a pool of worker processes.



- `Pool(processes=N)`: Creates a pool with N workers.
- `pool.map()`: Applies a function to an iterable, blocking until all results are ready (preserves order).
- `pool.apply_async()`: Submits a single task asynchronously, returning an `AsyncResult`.
- `pool.starmap()`: Like `map`, but unpacks arguments from iterables.

Now, let's see these theoretical topics in action.

- [04_process_communication_pool.py](file:///Users/vinod/Desktop/CDAC/20260927-hpc-python-parallel-programming/day2/demo/04_process_communication_pool.py)

Session 2 Summary

Concept	Purpose	Key Classes / Methods
Spawning	Start a new independent process	<code>Process</code> , <code>start()</code> , <code>join()</code>
Lifecycle	Monitor and kill processes	<code>terminate()</code> , <code>is_alive()</code> , <code>daemon=True</code>
Subclassing	Object-oriented process creation	<code>class MyProc(Process): def run(self): ...</code>
Queues	Safe multi-producer/multi-consumer communication	<code>Queue</code> , <code>put()</code> , <code>get()</code>
Pipes	Point-to-point communication	<code>Pipe</code> , <code>send()</code> , <code>recv()</code>
Synchronization	Protect shared resources	<code>Lock</code> , <code>Event</code> , <code>Semaphore</code>
Shared State	Share data between processes	<code>Value</code> , <code>Array</code> , <code>Manager</code>
Process Pools	Parallelize tasks over a data collection	<code>Pool</code> , <code>map()</code> , <code>apply_async()</code>

What's Coming Tomorrow (Day 3)

In our next sessions, we will scale up and change gears:

- **MPI with mpi4py:** Message Passing Interface for distributed computing across multiple nodes/computers!
- **Asynchronous programming with asyncio:** Concurrency for I/O-bound tasks in a single thread.
- **concurrent.futures:** High-level interfaces for thread and process pools.